PROFILE / BACKEND DEVELOPER

ABOUT ME

이벤트 처리의 상태와 책임 경계를 설계하고, 공유 상태 정합성과 비동기 실행 문제를 다뤄 온 백엔드 개발자



최승언

chygold06@gmail.com

GitHub · ChoiSeungeon ↗

BACKEND FOCUS

이벤트 처리
공유 상태 정합성
비동기 실행과 상태 전이

EDUCATION

2020.03 — 2026.02 · 졸업

순천향대학교
컴퓨터공학과
GPA **4.33 / 4.5**

RESEARCH / EXPERIENCE

2023.03 — 2025.06

OOPSLA Lab · 순천향대학교
학부연구생

CERTIFICATES

2025.12 **정보처리기사**
한국산업인력공단

2025.12 **SQLD**
한국데이터산업진흥원

TRAINING

2025.08 — 2026.03

유레카 SW 교육 3기
멀티캠퍼스 · 백엔드 과정

2026.07.09 — 교육 중

삼성 청년 SW·AI 아카데미
SSAFY 16기

AWARDS

2026.01.27 **K-Digital Training 과정종합 프로젝트경진대회 우수상**
멀티캠퍼스

2025.05.24 **우수 논문 및 학술 발표 동상**
순천향대학교 공과대학

PROJECT EVOLUTION **FIRE-SMOKE** 처리 단계 분리 → **TR1L** 상태 · 실행 제어 → **DABOM** 정합성 · 복구 인계

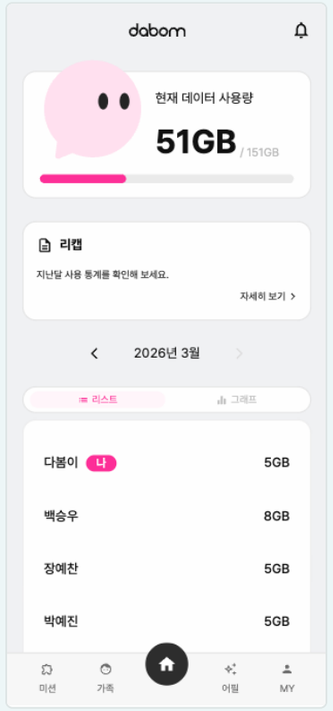
프로젝트를 거치며 처리 단계를 나누는 것에서 시작해, 상태와 실행을 제어하고, 실패 이후 복구 가능한 상태를 남기는 구조까지 발전시켜 왔습니다.

DABOM 다봄

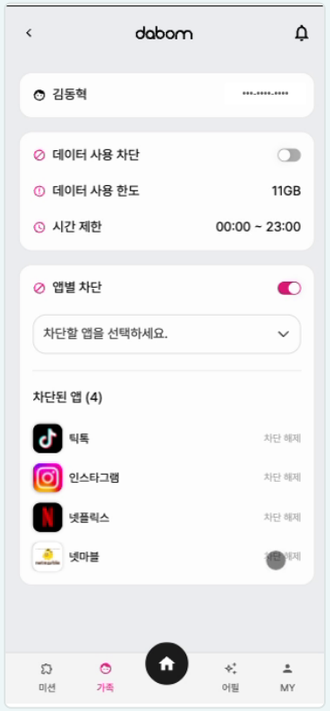
가족이 함께 쓰는 데이터의 사용량과 잔여량을 확인하고, 구성원별 사용 규칙과 정책 변경 요청을 관리하는 서비스

◈ **LG U+ URECA 3기 · 최종 프로젝트** 2026.02–2026.03 · 팀 7명 (FE 2 / BE 5)

서비스 화면



사용 현황 확인



한도 · 시간 · 앱 제어



정책 변경 요청

Java · Spring Boot · Kafka · Redis · Lua · MySQL

FOR FAMILIES

공유 데이터를 쓰는 가족 구성원과 사용 규칙을 설정하는 보호자를 위한 서비스입니다.

가족 공동 잔여량과 개인별 소비를 함께 보고, 구성원마다 필요한 이용 범위를 조율합니다.

CORE FEATURES

사용량 / 잔여량	가족과 구성원별 현황 확인
사용 정책	한도·이용 시간·앱 차단 설정
정책 조율	변경량과 사유를 담아 이의제기
월간 리캡	지난 사용 패턴 돌아보기

> 현황 확인 → 구성원 정책 설정 → 필요 시 변경 요청
> 사용 이벤트는 정책 판단·정산·알림 흐름으로 이어집니다.

통신사 가족 데이터 공유 모델을 참고한 가족 데이터 관리 서비스

MY ROLE

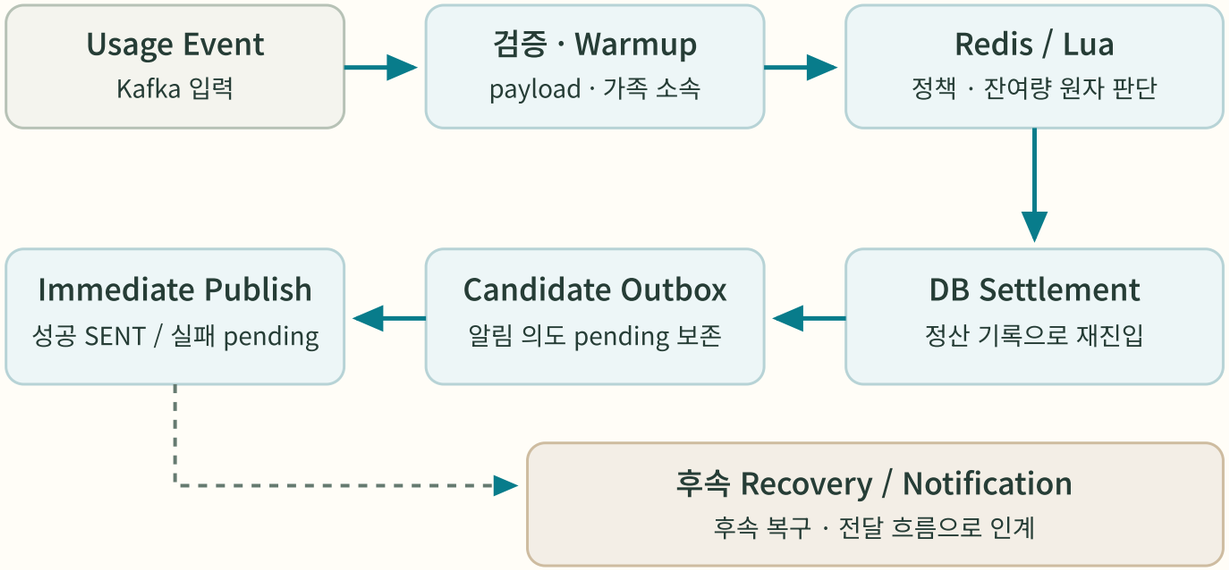
Usage 처리 영역 구현과 설계·통합 주도
lib-kafka 구현 · Family API 구현

핵심 성과

- 판단 · 정산 · 발행 인계의 3개 완료 지점을 분리한 Usage 흐름 구성
- Kafka 공통 메시징 절차를 lib-kafka로 분리·구현

Usage 이벤트를 판단·정산·발행으로 연결

사용량 처리 영역을 구현하고, 해당 영역의 설계·통합을 주도했습니다.



개인 구현 범위를 실선으로, 후속 복구·전달 인계를 점선으로 표시했습니다.

IMPLEMENTED OUTPUT

상태 판단, 영속 정산, 알림 의도 보존과 즉시 발행을 하나의 Usage 처리 흐름으로 구성했습니다.

MY IMPLEMENTATION

payload와 가족 소속을 검증하고, Redis warmup 이후 Lua 판단을 실행합니다. 판단 결과를 직접 DB 정산으로 연결했습니다.

NOTIFICATION HANDOFF

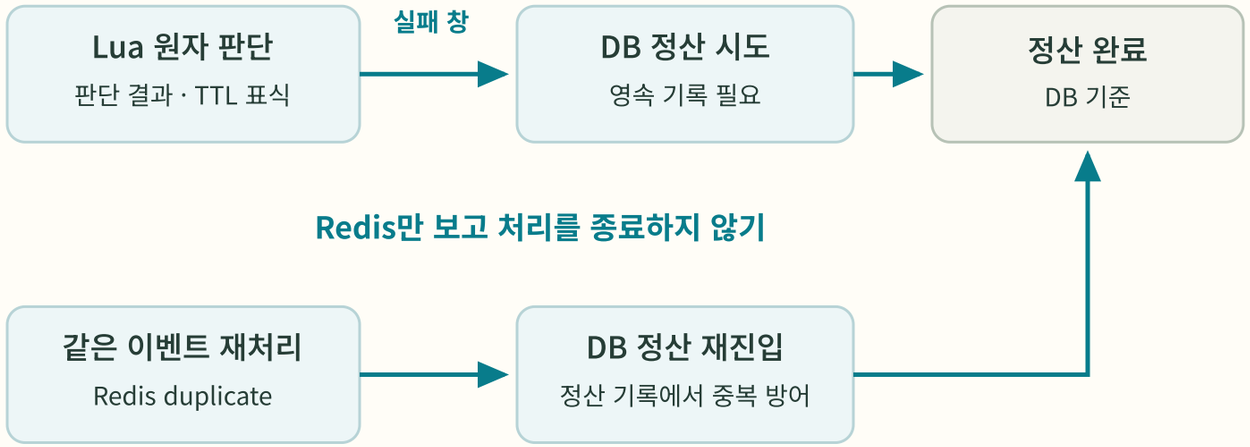
알림 후보를 Outbox에 남기고 즉시 발행합니다. 발행 성공은 SENT, 실패는 pending으로 유지해 후속 복구가 이어받도록 했습니다.

SHARED MODULE / API

메시징 절차를 묶는 lib-kafka와 가족·구성원 조회/관리 API도 구현했습니다. 상세에서는 Usage 정산과 발행 실패 문제에 집중합니다.

Redis 판단 ≠ DB 정산

Redis duplicate는 이미 내린 판단을 뜻합니다. 영속 정산까지 끝났다는 뜻은 아닙니다.



설계상 실패 조건과 재처리 흐름

RESULT

Redis 판단과 영속 정산의 완료 상태를 구분하고, duplicate에도 DB 정산에 재진입하는 처리 구조를 구현했습니다.

문제와 원인

Redis 판단 이후 DB 처리에 실패하면, 다음 입력은 duplicate가 되더라도 정산 기록이 없을 수 있습니다. 두 저장소는 한 트랜잭션으로 완료되지 않습니다.

선택한 방식

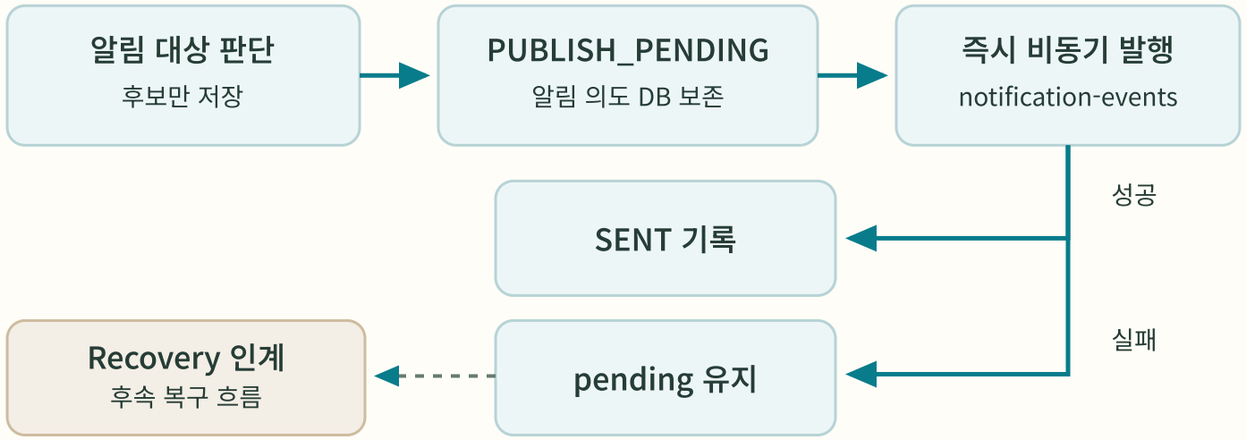
duplicate를 전체 처리 종료 조건으로 삼는 대신 DB 정산 재진입을 허용했습니다. Consumer 재처리와 Usage 내부 재진입을 구분해 정책을 정리했습니다.

TRADE-OFF / NEXT STEP

Redis 판단과 DB 정산을 분리해 재처리 가능한 완료 지점을 구분했습니다. 대신 정산 여부를 확인하는 DB 접근이 추가됩니다. 다음 단계에서는 DB 정산 기록을 기준으로 TTL 만료·실패 창에서의 멱등성 검증을 강화할 수 있습니다.

알림 발행 실패를 복구 가능한 상태로 남기기

정산 이후 발행이 실패해도, 알림 의도를 먼저 보존해 다음 처리가 이어받게 했습니다.



RESULT

후보 Outbox 저장·즉시 발행·성공/실패 상태 인계를 Usage 영역에 구현했습니다. 복구가 필요한 의도를 pending으로 남깁니다.

문제 상황

DB 정산과 메시지 발행은 다른 작업입니다. 정산 뒤 발행 실패를 처리하려면 무엇을 다시 보낼지 남아 있어야 합니다.

선택 이유와 구현

정산은 DB에 직접 반영하고, 알림 대상만 Outbox에 저장하도록 설계했습니다. 불필요한 쓰기를 줄이면서 발행 후보의 의도를 보존하는 선택입니다.

TRADE-OFF / NEXT STEP

발행 실패 시 PUBLISH_PENDING을 유지해 후속 복구의 인계 지점을 만들었습니다. DB 정산과 알림 의도 저장은 하나의 원자적 완료 경계가 아니므로, 다음 단계에서는 두 상태 사이의 실패 창을 좁히는 구조로 확장할 수 있습니다.

EVOLUTION

실행 이후 실패를 남기는 문제를, DABOM에서는 알림 의도 저장 → 즉시 발행 → pending 유지 → Recovery Handoff의 명시적 상태·인계 흐름으로 발전시켰습니다.

PROJECT 02 / TR1L

TR1L

월별 통신 요금을 정산하고 고객별 청구서를 생성해, 발송 정책에 맞춰 Mock Email·SMS로 전달하는 팀 프로젝트

◈ LG U+ URECA 3기 · 종합 프로젝트 참여 2026.01 · 팀 6명 · 모의 채널 발송

청구 생성부터 결과 상태까지

01 월 정산 · 고객별 청구서 생성
정산 스냅샷 → 청구 산출물 → 발송 대상 READY

02 발송일 · 수신거부 시간대 · 채널 정책
관리자 정책을 바탕으로 대상 선별 및 요청 이벤트 발행

03 Mock Email / SMS Delivery
청구서를 읽어 모의 발송 실행

04 결과 상태 관리
SENT 선점 이후 SUCCEED / FAILED 기록

FOR BILLING OPERATIONS

청구 대상 고객에게 전달할 산출물을 만들고, 운영자가 발송 정책과 실행 요청을 관리합니다.

정산·파일 생성과 I/O 발송을 분리해, 발송 집중과 지연을 서로 다른 처리 단계에서 다룹니다.

CORE FEATURES

고객별 월 정산·청구서 생성
발송일·수신거부 시간대·채널 정책 관리
관리자 수동 실행 요청과 상태 기반 모의 발송

> 정책 설정 → 월 정산·청구서 생성 → 대상 선별
> → Mock 발송 → 결과 상태 기록

통신 요금 정산과 비동기 발송 흐름을 구현한 팀 프로젝트

MY ROLE

Delivery Consumer / Worker와 상태 처리
실행 구조의 세 가지 대안을 직접 제안

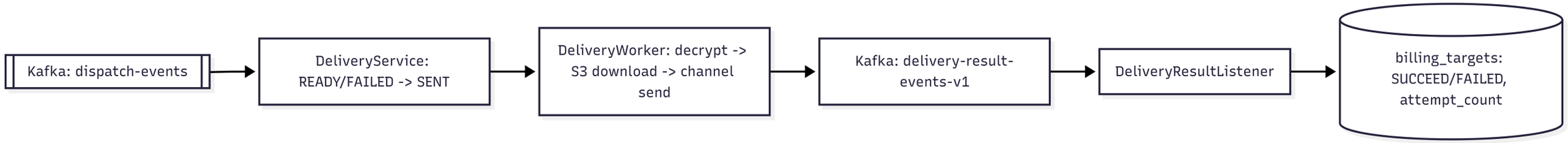
핵심 성과

- Delivery 선점·실행·결과 소비 구조 구현
- 처리 적체 위치와 상태 처리 순서를 개선하기 위한 3가지 실행 구조 제안·적용
- 프로젝트경진대회 우수상
  K-Digital Training 과정종합 · 팀 프로젝트 수상

Java Spring Kafka PostgreSQL S3

SENT는 완료가 아니라 실행 선점

상태 선점, 모의 발송 실행, 결과 소비를 분리한 Delivery 흐름을 구현했습니다.



Delivery Consumer Flow · 선점과 완료 상태의 차이

CONDITIONAL STATE TRANSITION

상태 변화	의미
READY / FAILED → SENT	조건을 충족한 대상의 실행 선점 선점 결과를 확인한 뒤 Worker에 제출
SENT → SUCCEED	모의 발송 결과를 소비해 성공 기록
SENT → FAILED	실패 결과 및 시도 횟수 기록

1 요청 Consumer
요청 파싱 → 조건부 선점 → Worker 제출

2 Delivery Worker
수신 정보 복호화 → S3 산출물 → Mock Email / SMS

3 결과 Consumer
결과 이벤트 수신 → 대상 상태 갱신

RESULT
요청 소비와 I/O 실행, 결과 상태 처리를 구분하는 Delivery 구조를 연결했습니다.

NEXT CHALLENGE 실패를 상태로 남기고, 후속 복구를 어떻게 인계할 것인가?
→ DABOM · Outbox · PUBLISH_PENDING · Recovery Handoff

실행 여력이 없을 때 어디에서 기다릴 것인가

소비와 I/O 처리 속도가 다를 때의 적체 위치, 그리고 상태 갱신의 순서를 비교했습니다.

메모리 내부 큐에 작업을 계속 쌓으면 대기 작업이 application 안에 누적됩니다. **Consumer / Worker 분리, 대기 큐 0 + Block, SENT 상태 갱신 비동기 제거**를 직접 제안했습니다.

PERSONAL PROPOSAL / 01

Consumer / Worker 분리

WHY

메시지 소비와 다운로드·발송 I/O 실행의 역할을 분리합니다.

EXPECTED EFFECT

Consumer는 선점과 제출, Worker는 발송 실행에 집중합니다.

PERSONAL PROPOSAL / 02

대기 큐 0 + Block

WHY

내부 대기 작업의 누적을 피하고, 처리 여력이 없으면 Consumer가 기다리게 합니다.

EXPECTED EFFECT

Kafka lag와 입력 대기로 적체 위치를 명확하게 드러냅니다.

PERSONAL PROPOSAL / 03

SENT 상태 갱신 비동기 제거

WHY

상태 갱신과 Worker 제출의 실행 순서를 명확하게 유지합니다.

EXPECTED EFFECT

상태 갱신 결과를 확인한 다음 Worker에 제출하는 순서를 명확히 합니다.

DECISION CRITERIA

처리량뿐 아니라 작업의 대기 위치와 상태 갱신·제출 순서를 명확히 하는 것을 기준으로 대안을 비교했습니다.

RESULT 세 가지 개선안을 바탕으로 작업의 대기 위치와 상태 갱신·제출 순서를 명확히 하는 실행 구조를 담당 영역에 적용했습니다.

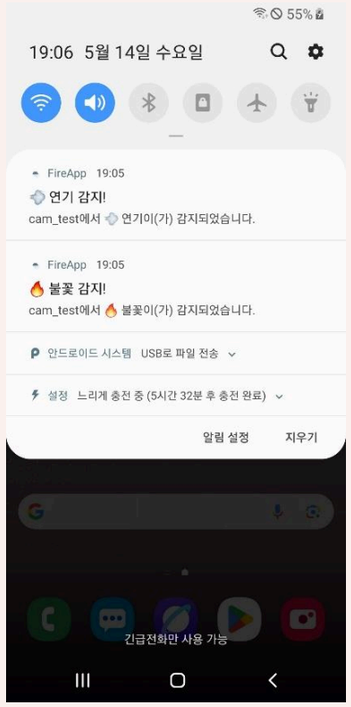
FIRE-SMOKE

CCTV 영상의 화재·연기 후보를 탐지하고, 저장한 이력과 알림을 모바일에서 확인하도록 연결한 1인 학사 졸업논문 시스템

◈ **순천향대학교 · 학사 졸업논문** 2025.03–2025.05 · 1인 프로젝트 · 연구 환경



화재·연기 탐지 결과



Android FCM 알림

FOR CCTV MONITORING

CCTV를 확인하는 공간 관리자를 연구상 사용자로 설정했습니다.

영상 속 탐지 후보를 이력과 알림으로 연결해, 관리자가 모바일에서 확인할 수 있는 흐름을 구성합니다.

CORE FEATURES

영상 입력 → 화재·연기 후보 추론
탐지 결과 저장 → Android 목록·상세 조회
유효 label·설정·토큰 조건에서 FCM 알림 발신

> 영상 입력 → 탐지 → 결과 저장 → API → Android
> 저장 이후 조건부 FCM 알림으로 분기

MY ROLE

1인 시스템 설계·구축·통합
외부 YOLO 모델 활용 · AI 보조 개발

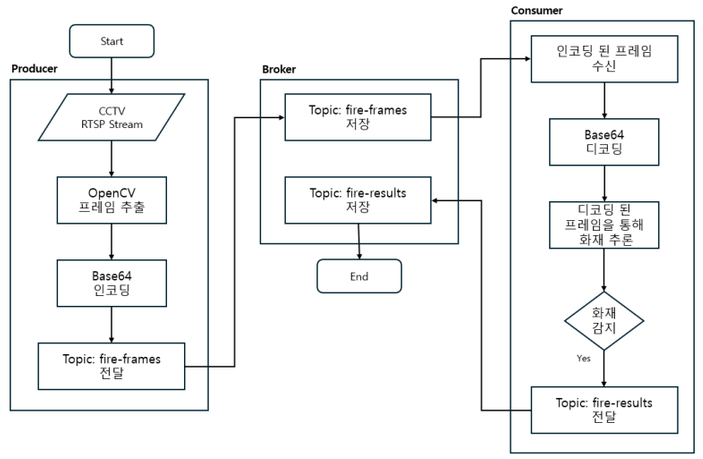
핵심 성과

- 1인 영상 → 추론 → 저장 → API → Android 시스템 구축
- mAP@0.5 0.899
  화재·연기 2클래스 모델 평가
- 우수 논문 및 학술 발표 동상
  순천향대학교 공과대학 · 학사논문 수상

Python · OpenCV · Kafka · YOLOv5s · SQLite · FastAPI · Android · FCM

탐지 결과를 저장·조회·앱으로 연결

입력 프레임과 분석 결과를 분리하고, 저장된 이력을 조회와 알림으로 연결하는 구조를 구성했습니다.



영상 입력–탐지 결과 메시지 흐름

1 프레임 / 추론

OpenCV → fire-frames → YOLO → fire-results

2 저장 / 알림

Saver → SQLite
유효 조건에서 FCM 발신

3 조회 / Android

FastAPI가 저장된 이력을 읽어 목록·상세로 연결

문제 상황

탐지 결과가 모델 안에만 머무르면 이력 조회와 모바일 확인으로 이어지지 않습니다. 입력·추론·결과 저장의 작업을 나눠 연결할 필요가 있습니다.

선택한 방식

Kafka로 프레임과 결과를 전달하고, 별도 Saver에서 저장·조건부 알림을 수행하도록 구성했습니다. FastAPI는 같은 SQLite의 이력을 조회합니다.

RESULT

영상 입력 → 추론 → 결과 저장 → API 조회 → Android 연계로 이어지는 연구 시스템을 구축했습니다.

NEXT STEP

알림 실패를 내구성 있게 남기고, 실패한 알림을 다시 처리할 수 있도록 재시도 상태를 관리하는 구조로 확장할 수 있습니다.

NEXT FOCUS

처리 단계 분리에서, 각 단계의 실행 상태와 처리 순서를 더 명확히 제어하는 문제로 관심이 이어졌습니다.

→ TR1L · SENT 상태 · Consumer / Worker